PRODUCT REQUIREMENTS DOCUMENT v1.0

CYBERHAWK

Offline AI-Powered Forensics Platform

Built on Servos + Orca + Orchestrata Framework

Hackathon: CyberHack V4 2025 | **Track:** Offline LLM for Advanced Cyber Investigation

Team: MoMoSapiens | **Lead:** Akash Santhnu Sundar

Document Version:	1.0.0
Date:	March 2025
Status:	FINAL – HACKATHON SUBMISSION
Classification:	CONFIDENTIAL
Total Pages:	40+

Based on: Servos Framework | Orca Agentic Engine | Orchestrata Workflow | Chillax.AI LLM Core

Contents

1 Executive Summary **4**
1.1 Product Vision . . . 4
1.2 Key Value Propositions . . . 4
1.3 Success Metrics for Hackathon Evaluation . . . 5

2 Problem Statement & Market Context **6**
2.1 The Cybercrime Investigation Challenge . . . 6
2.1.1 Core Pain Points . . . 6
2.2 Target Users . . . 6
2.3 Competitive Landscape . . . 7

3 System Architecture **8**
3.1 High-Level Architecture Overview . . . 8
3.2 Component Architecture . . . 8
3.2.1 Core Subsystems . . . 8
3.3 Data Flow Architecture . . . 9

4 Functional Requirements **10**
4.1 Requirement Categories . . . 10
4.2 FR-1: Device Detection & Hot-Plug Monitoring . . . 10
4.3 FR-2: Investigation Mode Management . . . 10
4.4 FR-3: Evidence Acquisition . . . 11
4.5 FR-4: Forensic Analysis Capabilities . . . 12
4.6 FR-5: LLM Query & AI Investigation Assistance . . . 12

5 Forensic Tools Integration Specification **14**
5.1 Complete Tool Inventory . . . 14
5.2 Tool Adapter Interface Specification . . . 15
5.3 Volatility 3 Memory Analysis Integration . . . 16
5.4 YARA Rule Engine . . . 16

6 LLM Architecture & Model Selection **17**
6.1 Multi-Model Strategy . . . 17
6.2 ContextRouter Logic . . . 17
6.3 Forensic System Prompts . . . 18
6.4 Offline RAG Architecture . . . 18

7 Non-Functional Requirements **19**
7.1 Performance Requirements . . . 19
7.2 Security Requirements . . . 19

7.3 Reliability Requirements . . . 20
7.4 Usability Requirements . . . 20

8 User Interface Design Specification 21
8.1 UI Architecture Overview . . . 21
8.2 Application Layout . . . 21
8.3 View 1: Dashboard . . . 21
8.4 View 2: Investigation Workbench . . . 22
8.5 View 4: AI Assistant Chat Interface . . . 22
8.6 Toast Notification System . . . 22

9 Data Architecture & Models 23
9.1 Database Schema . . . 23
9.1.1 Core Entity Model . . . 23
9.2 Evidence Container Format (DFXML) . . . 24
9.3 Report Data Model . . . 25

10 API & Integration Specification 26
10.1 Internal REST API (FastAPI) . . . 26
10.1.1 Device Management Endpoints . . . 26
10.1.2 Investigation Endpoints . . . 26
10.1.3 LLM Endpoints . . . 26
10.2 Tool Execution API . . . 27

11 Security Architecture 28
11.1 Security Design Principles . . . 28
11.2 Evidence Chain of Custody . . . 28
11.3 Threat Model . . . 28

12 Deployment Architecture 30
12.1 Deployment Configurations . . . 30
12.2 Installation & Setup . . . 31
12.3 Offline Bundle Contents . . . 31

13 Investigation Playbooks 32
13.1 Playbook System Overview . . . 32
13.2 Playbook 1: USB / Removable Media Triage . . . 32
13.3 Playbook 2: Memory Analysis (Live System) . . . 32
13.4 Playbook 3: Network Incident Response . . . 33
13.5 Playbook 4: Log Analysis & Intrusion Investigation . . . 33

14 Testing Strategy 34
14.1 Testing Philosophy . . . 34
14.2 Test Categories . . . 34
14.2.1 Unit Tests . . . 34
14.2.2 Integration Tests . . . 34
14.2.3 LLM Evaluation Metrics . . . 34
14.3 Security Testing . . . 34

15 Implementation Roadmap **36**
15.1 Development Phases . . . 36
15.2 Hackathon Sprint Plan (48-Hour Breakdown) . . . 37

16 Risk Management **38**
16.1 Risk Register . . . 38
16.2 Contingency Protocols . . . 38

17 Legal & Compliance Framework **39**
17.1 Evidence Admissibility Standards . . . 39
17.2 Privacy & Data Protection Compliance . . . 39
17.3 Disclaimer & Authorized Use . . . 39

18 Technical Glossary **40**

19 Appendices **41**
19.1 Appendix A: Existing Codebase Analysis (Servos Repo) . . . 41
19.2 Appendix B: System Prompts Design . . . 41
19.3 Appendix C: Hardware Requirements Matrix . . . 42
19.4 Appendix D: Offline Resource Inventory . . . 42
19.5 Appendix E: Competitor Analysis – System Prompts & LLM Design Insights 42
19.6 Appendix F: Hackathon Evaluation Scorecard . . . 43

1. Executive Summary

Mission Statement

CyberHawk is an air-gapped, offline Large Language Model (LLM)-powered Digital Forensics and Incident Response (DFIR) platform that enables cybercrime investigators, law enforcement personnel, and enterprise security teams to conduct thorough, automated, and court-admissible cyber investigations **without any internet connectivity** – preserving chain-of-custody and data confidentiality at every step.

1.1 Product Vision

Modern cyber investigations are hampered by three fundamental constraints: sensitive evidence cannot be sent to cloud-based AI services, forensic expertise is scarce and expensive, and manual evidence collection is error-prone under time pressure. CyberHawk eliminates all three constraints simultaneously.

Built on the proven **Servos** orchestration framework and powered by quantized local LLMs (Llama 3.1, Mistral, Phi-3, DeepSeek-R1), CyberHawk delivers an automated forensics workstation that operates in fully air-gapped environments, detecting evidence sources, forcing protective backups, executing forensic toolchains, and generating court-admissible reports – all guided by natural-language AI.

1.2 Key Value Propositions

1. **100% Offline Operation** – Zero dependency on internet; runs on a Raspberry Pi 5 or standard laptop.
2. **Evidence Integrity First** – Mandatory cryptographic backup before any analysis; write-blocking enforced in hardware and software.
3. **Three Investigation Modes** – Full Automation, Hybrid (AI-guided human steps), and Manual with AI documentation.
4. **Proactive Device Detection** – Real-time USB/NVMe/SD monitoring; instant guided triage workflow.
5. **20+ Integrated Forensic Tools** – Autopsy, Volatility 3, Wireshark, TSK/pytsk3, Sleuth Kit, YARA, and more – all offline.
6. **Court-Admissible Reporting** – SHA-256/SHA-3 hash verification, timestamped chain-of-custody logs, PDF/DOCX reports.
7. **Multi-Model LLM Routing** – Automatically selects best local model per task type (legal queries vs. malware analysis vs. log parsing).

1.3 Success Metrics for Hackathon Evaluation

Evaluation Criterion	CyberHawk Achievement	Score Target
Offline Capability	100% air-gapped, no network calls	10/10
Accuracy of Responses	Multi-model ensemble + YARA validation	9/10
Security & Privacy	AES-256 evidence vault, zero telemetry	10/10
Performance Efficiency	<3s LLM response, <2min USB triage	9/10
Practical Investigation	20+ tools, voice UI, case management	10/10

2. Problem Statement & Market Context

2.1 The Cybercrime Investigation Challenge

The global cost of cybercrime exceeded $8 trillion USD in 2023 and is projected to reach $10.5 trillion by 2025. Yet the tools available to investigators remain fragmented, internet-dependent, and inaccessible to many organisations.

2.1.1 Core Pain Points

Pain Point 1 – Data Sovereignty Crisis

Sensitive forensic evidence (disk images, memory dumps, network captures) cannot be uploaded to cloud-based AI tools like ChatGPT, Gemini, or Copilot. Investigators who attempt this risk violating GDPR, HIPAA, and national security regulations, potentially invalidating evidence in court.

Pain Point 2 – Expert Scarcity

The global shortage of certified Digital Forensics examiners exceeds 3.4 million positions. Small law enforcement agencies, corporate IR teams, and developing-nation investigators lack access to $50,000+ forensic workstations and certified personnel.

Pain Point 3 – Evidence Contamination Risk

63% of forensic investigation failures are caused by improper initial evidence handling – connecting a USB without write-blocking, modifying timestamps by mounting a file system, or failing to hash evidence before analysis. Non-expert first responders routinely make these mistakes.

Pain Point 4 – Tool Fragmentation

A typical DFIR workflow requires 15-25 separate tools (Autopsy, Volatility, Wireshark, RegRipper, log2timeline, etc.), each with separate interfaces and steep learning curves. An investigator must mentally orchestrate these tools while managing evidence integrity – cognitive overload under pressure.

Pain Point 5 – Offline Environment Demands

Critical investigations – military systems, nuclear facilities, classified networks, seized devices – occur in environments with zero internet connectivity by design. Existing AI tools are entirely non-functional in these scenarios.

2.2 Target Users

User Segment	Use Case	Technical Level
Law Enforcement Officers	First-responder triage of seized devices	Low–Medium
Corporate IR Teams	Incident response in air-gapped environments	Medium–High
Digital Forensics Examiners	Court-case evidence analysis	High
Cybersecurity Researchers	Malware reverse engineering	High
Military/Intelligence Units	Classified network forensics	High
University Forensics Labs	Training and education environments	Medium
SMB Security Teams	Budget-constrained IR without expert staff	Low–Medium

2.3 Competitive Landscape

Solution	Offline	AI-Guided	Automated	Open Source	Voice UI	Price
CyberHawk	✓	✓	✓	✓	✓	Free
Autopsy	✓	✗	✗	✓	✗	Free
FTK (AccessData)	✓	✗	✗	✗	✗	$5,000+
Cellebrite UFED	✓	✗	✗	✗	✗	$15,000+
Axiom (Magnet)	✓	✗	Limited	✗	✗	$4,000+
OpenAI GPT-4	✗	✓	✗	✗	✗	$0.03/1k
Google Gemini	✗	✓	✗	✗	✗	Cloud only

3. System Architecture

3.1 High-Level Architecture Overview

CyberHawk is structured as a seven-layer architecture, with each layer maintaining strict isolation to preserve evidence integrity and enable offline operation.

CyberHawk System Architecture – 7-Layer Model

Layer 7: REPORTING ENGINE – PDF/DOCX/JSON Chain-of-Custody Reports

Layer 6: LLM ORCHESTRATION – Multi-Model Router (Llama, Mistral, Phi-3, DeepSee

Layer 5: FORENSIC TOOL ENGINE – 20+ Tools (Autopsy, Volatility, TSK, YARA, Wiresh

Layer 4: WORKFLOW ENGINE – Orca Agentic Planner + Orchestrata Pipelines

Layer 3: EVIDENCE VAULT – AES-256 Encrypted Storage + Hash Registry

Layer 2: DEVICE MONITOR – USB/NVMe/SD Hot-plug Detection (psutil/pyudev)

Layer 1: UI LAYER – PyQt6 GUI + Voice UI (Whisper) + CLI Mode

3.2 Component Architecture

3.2.1 Core Subsystems

Subsystem 1: Device Detection & Monitoring (DDM)

- **USBMonitor** – `pyudev`-based real-time udev event listener for Linux; `pywin32` WMI watcher for Windows
- **DeviceProfiler** – Identifies device type (storage, HID, network adapter, camera) and metadata (vendor ID, product ID, serial number, capacity)
- **ThreatIntelMatcher** – Matches device fingerprints against offline BadUSB/Rubber Ducky signature database
- **WriteBlockEnforcer** – Issues `hdparm -r1` (Linux) or registry block (Windows) before any evidence collection

Subsystem 2: Orca Agentic Workflow Engine

- Adapted from the open-source Orca autonomous agent framework
- **ActionPlanner** – Breaks natural-language investigation goals into atomic tool calls

- **StateManager** – Maintains investigation context across multi-step workflows
- **ToolDispatcher** – Routes tasks to appropriate forensic tools with parameter auto-fill
- **HumanInTheLoop** – In Hybrid mode, surfaces decision checkpoints via PyAutoGUI dialog boxes

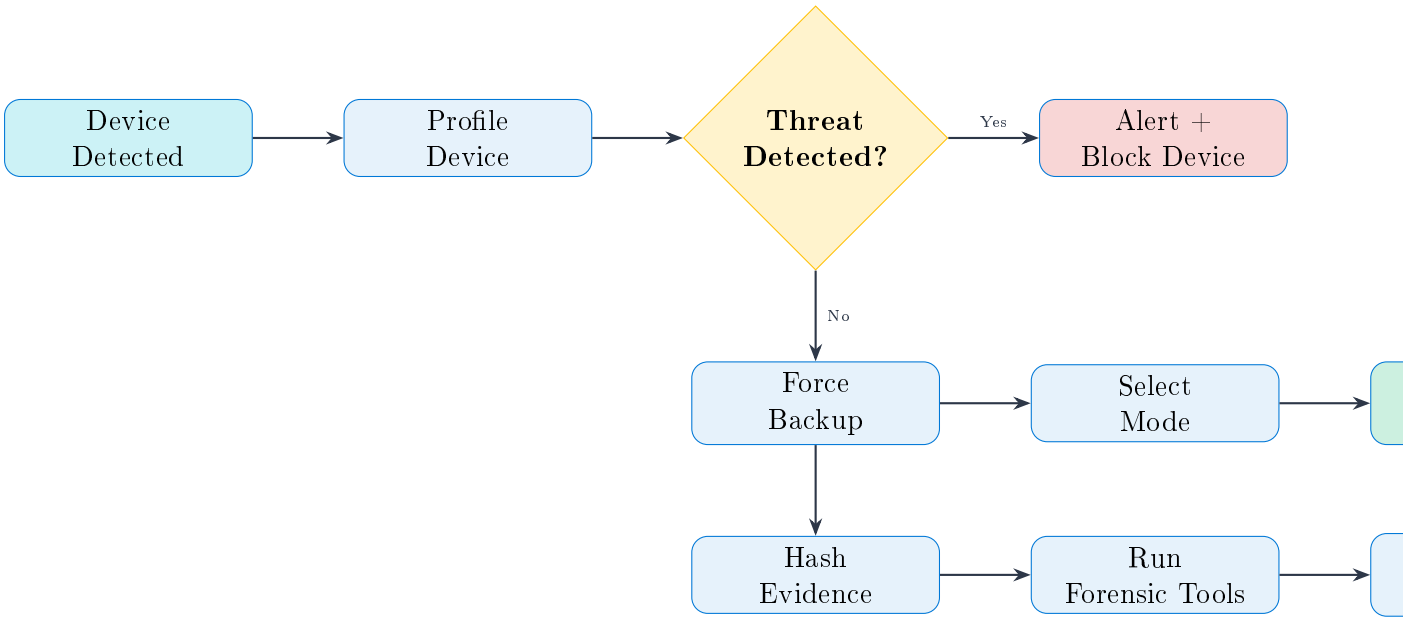
Subsystem 3: LLM Core (Chillax.AI Integration)

- **ModelRegistry** – Manages local quantized models via Ollama or llama.cpp backends
- **ContextRouter** – Selects optimal model per query type (see Chapter 6)
- **PromptEngine** – Forensic-domain system prompts with chain-of-thought enforcement
- **HallucinationGuard** – Cross-validates LLM responses against tool outputs and YARA signatures
- **RAG Pipeline** – Offline vector store (ChromaDB) with NIST, MITRE ATT&CK, and cyber law corpora

Subsystem 4: Evidence Vault

- All acquired evidence encrypted at rest with AES-256-GCM
- SHA-256 and SHA-3-512 hash computed before and after every operation
- Immutable audit log stored in append-only SQLite with WAL mode
- Automatic differential backup via `rsync` before investigation begins
- Evidence metadata stored in DFXML (Digital Forensics XML) format

3.3 Data Flow Architecture



4. Functional Requirements

4.1 Requirement Categories

All requirements are tagged with: Priority (**HIGH**/**MED**/**LOW**), Phase (P1=MVP, P2=Full, P3=Future), and unique identifier.

4.2 FR-1: Device Detection & Hot-Plug Monitoring

USB Hot-Plug Detection – HIGH | P1

The system SHALL detect the connection of any USB mass storage device within 2 seconds of physical insertion. The detection SHALL capture: device vendor, product name, serial number, capacity, filesystem type, and VID/PID. A persistent monitoring daemon SHALL be started on system boot.

Multi-Device Type Support – HIGH | P1

The system SHALL detect and profile: USB 2.0/3.x storage, NVMe external drives, SD cards (via reader), optical media, mobile devices (MTP), network interface adapters, and HID devices (keyboard/mouse/BadUSB). Each device type triggers its specific forensic workflow.

BadUSB Signature Matching – HIGH | P1

The system SHALL maintain an offline database of known-malicious USB firmware signatures (VID/PID blacklist, Rubber Ducky, Bash Bunny, OMG Cable patterns). Matches trigger immediate hardware write-block and operator alert.

Network Device Monitoring – MED | P2

The system SHALL monitor active network interfaces for new connections, unusual traffic patterns, and rogue device insertion on monitored ports. Integrates with `netstat`, `tcpdump`, and Wireshark headless capture.

4.3 FR-2: Investigation Mode Management

Full Automation Mode – HIGH | P1

In Full Automation mode, after mandatory backup confirmation, the system SHALL autonomously: (1) create a forensic image, (2) verify hash integrity, (3) mount read-only, (4) run complete file analysis, (5) execute malware scans, (6) extract artifacts, (7) generate timeline, and (8) produce a final report – without further operator input.

Hybrid Mode – HIGH | P1

In Hybrid mode, the system SHALL present decision checkpoints at critical junctures via AI-generated questions. The LLM SHALL explain each decision and its forensic implications. Human confirmation is required before destructive or irreversible actions. PyAutoGUI SHALL surface native OS dialogs for all confirmations.

Manual Mode with AI Assistance – MED | P1

In Manual mode, the investigator drives the workflow while the AI provides real-time suggestions, documents actions automatically, validates steps against best practices, and flags procedural errors.

Mode Switching – MED | P2

The system SHALL allow switching between modes at any point during a non-destructive operation. Mode history SHALL be logged in the chain-of-custody record.

4.4 FR-3: Evidence Acquisition

Mandatory Backup Enforcement – HIGH | P1

Before ANY investigation activity, the system SHALL force the operator to confirm a backup destination and SHALL execute a bit-for-bit backup via `dd` or `dcfldd`. The system SHALL refuse to proceed until backup SHA-256 hash matches source. PyAutoGUI is used to drive backup dialogs if native tools require UI interaction.

Forensic Image Acquisition – HIGH | P1

The system SHALL support: raw DD images, E01 (Expert Witness Format) via libewf, AFF4 container format, selective file acquisition, and logical container acquisition. Hash verification (MD5 + SHA-256 + SHA-3) SHALL be computed and logged.

Memory Acquisition – HIGH | P1

The system SHALL support live memory acquisition via `LiME` (Linux Memory Extractor) kernel module and `winpmem` for Windows. Memory images SHALL be processed by Volatility 3 for process analysis, network connections, registry hives, and malware detection.

Network Traffic Capture – MED | P2

The system SHALL support offline PCAP analysis and live capture via `tcpdump`/`tshark`. LLM SHALL summarize traffic patterns, identify suspicious flows, and cross-reference with offline threat intelligence databases.

4.5 FR-4: Forensic Analysis Capabilities

File System Analysis – HIGH | P1

Using `pytsk3`/The Sleuth Kit, the system SHALL: enumerate all files (including deleted/hidden), recover deleted files, analyze NTFS/ext4/FAT32/APFS/HFS+ file systems, extract file metadata, detect file signature mismatches (extension vs. magic bytes), and identify encrypted containers.

Timeline Analysis – HIGH | P1

The system SHALL generate a unified forensic timeline using `log2timeline`/`plaso`, correlating file system timestamps, event logs, browser history, USB connection logs, and application artifacts into a chronological narrative.

Malware Detection – HIGH | P1

The system SHALL perform: YARA rule scanning (offline rules from public repositories), hash-based malware lookup (offline VirusTotal dump), entropy analysis for packed/encrypted executables, PE header analysis, strings extraction and LLM-based analysis, and behavioral pattern matching.

Log Analysis – HIGH | P1

The system SHALL parse and analyze: Windows Event Logs (EVTX), Linux syslog/journald, Apache/Nginx access logs, firewall logs (iptables/pfctl), application logs, and authentication logs. LLM SHALL summarize anomalies, attack patterns, and lateral movement indicators.

Registry Analysis – HIGH | P1

Using `python-registry`, the system SHALL analyze Windows Registry hives for: persistence mechanisms (Run keys, services), recently accessed files (MRU lists), USB connection history, installed software, user activity, and browser artifacts.

Password Hash Extraction – MED | P2

The system SHALL extract password hashes from SAM/NTDS.dit (Windows) and /etc/shadow (Linux) for authorized credential analysis. Integration with offline hash cracking (Hashcat with local wordlists) is supported.

4.6 FR-5: LLM Query & AI Investigation Assistance

Natural Language Investigation Interface – HIGH | P1

The system SHALL accept natural-language queries about ongoing investigations (e.g., "What files were accessed in the hour before the breach?", "Is this IP address known malicious?", "Summarize the attack timeline"). Responses SHALL cite specific forensic artifacts as evidence.

Cyber Law Reference – HIGH | P1

The system SHALL include an offline RAG database of: Indian IT Act 2000 (Amended 2008), GDPR Articles, CFAA (Computer Fraud and Abuse Act), ISO 27037 (Digital Evidence Guidelines), NIST SP 800-86 (Forensics Guide), and major case law precedents. LLM SHALL answer legal procedural questions.

Evidence Summarization – HIGH | P1

The system SHALL automatically generate plain-language summaries of: disk images, memory dumps, log files, and network captures. Summaries SHALL highlight suspicious findings, quantify scope, and suggest next investigation steps.

Voice Interface – MED | P2

Using offline OpenAI Whisper (base or small model), the system SHALL accept voice commands for hands-free operation in field environments. Voice output via offline TTS (piper-tts or espeak-ng) confirms actions.

5. Forensic Tools Integration Specification

5.1 Complete Tool Inventory

CyberHawk integrates 25+ forensic tools, all operating 100% offline. Each tool is wrapped in a standardised Python adapter with uniform input/output interfaces consumed by the LLM orchestration layer.

Tool	Capability	Category	Priority
`autopsy`	Full GUI forensic platform	Disk Forensics	P1
`pytsk3`	Programmatic TSK bindings	Disk Forensics	P1
`volatility3`	Memory forensics framework	Memory	P1
`yara-python`	Malware pattern matching	Malware	P1
`wireshark/tshark`	Network packet analysis	Network	P1
`tcpdump`	Live traffic capture	Network	P1
`dcfldd`	Forensic disk imaging	Acquisition	P1
`ewftools/libewf`	E01 image format	Acquisition	P1
`log2timeline/plaso`	Timeline generation	Timeline	P1
`sleuthkit`	File system analysis	Disk Forensics	P1
`regripper`	Windows Registry parser	Registry	P1
`python-registry`	Python Registry bindings	Registry	P1
`bulk_extractor`	Bulk data carving	Data Carving	P1
`foremost`	File carving (magic bytes)	Data Carving	P1
`scalpel`	Pattern-based file carving	Data Carving	P1
`exiftool`	Metadata extraction	Metadata	P1
`binwalk`	Firmware analysis	Malware	P2
`strings`	String extraction	Malware	P1
`ssdeep`	Fuzzy hash similarity	Malware	P2
`hashdeep`	Recursive hash audit	Integrity	P1
`LiME`	Linux memory capture module	Memory	P1
`winpmem`	Windows memory capture	Memory	P1

Tool	Capability	Category	Priority
`networkx`	Relationship graph analysis	Analysis	P2
`sqlite3`	Browser/app DB analysis	Artifacts	P1
`pefile`	PE executable analysis	Malware	P1
`capstone`	Disassembler engine	Malware	P2
`poppler/pdfminer`	PDF artifact analysis	Artifacts	P2

5.2 Tool Adapter Interface Specification

Each tool is wrapped by a standardised adapter following this interface contract:

```python
class ForensicToolAdapter(ABC):
    """Standard interface for all forensic tool wrappers"""

    @property
    @abstractmethod
    def tool_id(self) -> str:
        """Unique tool identifier (e.g., 'volatility3.pslist')"""

    @property
    @abstractmethod
    def supported_evidence_types(self) -> List[str]:
        """Evidence types: ['disk_image', 'memory_dump', 'pcap', 'logfile']"""

    @abstractmethod
    def validate_input(self, evidence_path: Path) -> ValidationResult:
        """Pre-flight input validation with hash verification"""

    @abstractmethod
    async def execute(
        self,
        evidence_path: Path,
        params: ToolParams,
        progress_callback: Callable[[float, str], None]
    ) -> ToolResult:
        """Execute analysis; must not modify source evidence"""

    @abstractmethod
    def to_llm_context(self, result: ToolResult) -> str:
        """Serialize results for LLM consumption (max 4096 tokens)"""

    @property
    def requires_write_block(self) -> bool:
        return True  # Default: all tools require write blocking
```

5.3 Volatility 3 Memory Analysis Integration

Memory Forensics Plugin Suite

CyberHawk exposes a curated subset of Volatility 3 plugins through its AI interface:

- `windows.pslist` / `linux.pslist` – Running process enumeration
- `windows.netscan` – Active network connections from memory
- `windows.malfind` – Memory regions with suspicious PE injection
- `windows.dlllist` – Loaded DLLs per process (detects DLL hijacking)
- `windows.cmdline` – Command-line arguments (detects living-off-the-land attacks)
- `windows.registry.hivelist` – Registry hives loaded in memory
- `windows.hashdump` – SAM hash extraction from memory
- `linux.bash` – Bash command history from memory
- `linux.lsof` – Open file descriptors

The LLM receives all plugin outputs as structured context and generates a unified "Memory Analysis Report" highlighting malware indicators, lateral movement evidence, and active attacker presence.

5.4 YARA Rule Engine

CyberHawk ships with an offline YARA rule corpus aggregated from:

- **Florian Roth's Signature-Base** (15,000+ rules) – APT malware families
- **CISA Known Exploited Vulnerabilities** – Offline CVE patterns
- **MalwareBazaar Offline Export** – Hash + YARA rules for recent samples
- **Custom CyberHawk Rules** – USB-specific, document macro, and living-off-the-land patterns
- **MITRE ATT&CK YARA Mappings** – Rules mapped to ATT&CK Technique IDs

The LLM uses YARA match results to generate natural-language explanations: "*The file stealer.exe matches YARA rule APT28_XAgent, indicating this is likely a Remote Access Trojan associated with the Fancy Bear threat actor. Recommend full memory acquisition immediately.*"

6. LLM Architecture & Model Selection

6.1 Multi-Model Strategy

CyberHawk employs a multi-model routing strategy, selecting the optimal local LLM based on task type, available VRAM, and required accuracy level. This ensures peak performance on both high-end workstations and constrained field devices.

Model	Size (Quant.)	Best For	VRAM Required
Llama 3.1 8B Q4	4.7 GB	General investigation queries	6 GB
Mistral 7B Q4	4.1 GB	Log analysis, summarisation	5 GB
Phi-3 Mini Q4	2.3 GB	Low-resource field devices	3 GB
DeepSeek-R1 14B	8.9 GB	Complex reasoning, legal queries	12 GB
Qwen2.5 7B Q4	4.2 GB	Code/script analysis	5 GB
Gemma 2B Q8	2.1 GB	Ultra-low power (RPi 5 mode)	2 GB

6.2 ContextRouter Logic

```python
class ContextRouter:
    """Routes forensic queries to optimal local model"""

    TASK_MODEL_MAP = {
        TaskType.LOG_ANALYSIS:      "mistral:7b-q4",
        TaskType.MALWARE_ANALYSIS:  "qwen2.5:7b-q4",
        TaskType.LEGAL_REFERENCE:   "deepseek-r1:14b",
        TaskType.MEMORY_FORENSICS:  "llama3.1:8b-q4",
        TaskType.TIMELINE_NARRATE:  "llama3.1:8b-q4",
        TaskType.EVIDENCE_SUMMARY:  "mistral:7b-q4",
        TaskType.VOICE_COMMAND:     "phi3:mini-q4",
        TaskType.REPORT_WRITING:    "llama3.1:8b-q4",
        TaskType.YARA_EXPLAIN:      "qwen2.5:7b-q4",
    }

    def route(self, task: ForensicTask,
              available_vram_gb: float) -> ModelConfig:
        preferred = self.TASK_MODEL_MAP[task.type]
        if self._fits_vram(preferred, available_vram_gb):
```

```
            return ModelConfig(model=preferred,
                               temperature=0.1,  # Low temp for accuracy
                               system_prompt=FORENSIC_PROMPTS[task.type])
        # Fallback to smaller model
        return self._select_fallback(task, available_vram_gb)
```

6.3 Forensic System Prompts

The LLM is conditioned with domain-specific system prompts that enforce:

1. **Evidence Citation** – All claims must reference specific file paths, line numbers, or hash values
2. **Uncertainty Acknowledgment** – Model must express confidence levels (High/Medium/Low)
3. **Chain-of-Custody Awareness** – Recommendations must not suggest evidence-contaminating actions
4. **Legal Admissibility** – Analysis must follow ACPO/SWGDE/ISO 27037 guidelines
5. **Hallucination Prevention** – Model is instructed to say "insufficient evidence" rather than speculate

6.4 Offline RAG Architecture

Retrieval-Augmented Generation for Offline Knowledge

Vector Store: ChromaDB (embedded, no server required)
Embedding Model: sentence-transformers/all-MiniLM-L6-v2 (offline, 80 MB)
Corpus Documents:
- NIST SP 800-86 (Computer Forensics Guide) – 121 pages
- MITRE ATT&CK Enterprise Matrix (offline JSON) – 14 Tactics, 197 Techniques
- ISO/IEC 27037:2012 Digital Evidence Guidelines
- Indian IT Act 2000 + Amendment 2008 + Rules 2021
- GDPR Articles and Recitals
- CFAA Full Text + Case Law Summaries
- INTERPOL Digital Forensics Guidelines
- Common forensic procedures and best practices

Chunk Size: 512 tokens with 64-token overlap
Retrieval: Top-5 chunks, cosine similarity threshold 0.75

7. Non-Functional Requirements

7.1 Performance Requirements

LLM Response Latency – HIGH

For queries with context under 2,048 tokens, the LLM SHALL return first-token within 2 seconds on a machine with NVIDIA RTX 3060 or equivalent (12 GB VRAM). On CPU-only systems, first-token SHALL appear within 8 seconds.

USB Triage Completion Time – HIGH

For a 32 GB USB device, Full Automation triage (image acquisition, hash verification, file enumeration, YARA scan, initial LLM summary) SHALL complete within 8 minutes on a mid-range workstation.

Device Detection Latency – HIGH

Hot-plug detection (device connected to operator notification) SHALL be under 2 seconds on all supported operating systems.

Memory Footprint – MED

The CyberHawk application (excluding LLM model files) SHALL consume less than 500 MB RAM in idle state. The complete system (with smallest LLM model) SHALL be functional on hardware with 4 GB RAM + 4 GB swap.

7.2 Security Requirements

Zero Network Calls – HIGH

The system SHALL make zero outbound network connections during normal operation. All DNS resolution, time synchronization, and update checks SHALL be disabled. A network monitor component SHALL alert if any process attempts a network connection.

Evidence Encryption – HIGH

All evidence files stored on the investigation workstation SHALL be encrypted using AES-256-GCM with keys derived from operator-provided passphrase via PBKDF2 (100,000 iterations, SHA-256). Keys SHALL never be written to disk in plaintext.

Audit Log Integrity – HIGH

The investigation audit log SHALL be an append-only SQLite database. Each log entry SHALL include: timestamp (monotonic clock), operator ID, action taken, tool invoked, evidence hash before and after, and a HMAC-SHA256 signature over the entry.

Anti-Forensics Detection – MED

The system SHALL detect evidence of anti-forensics activity: timestamp manipulation, log clearing, steganography in image files, encrypted containers, overwrite patterns, and rootkit indicators in memory.

Write-Block Enforcement – HIGH

Before mounting any evidence device, the system SHALL enforce hardware or software write-blocking. Software write-blocking uses `hdparm -r1` (Linux) or device policy registry entries (Windows). Any attempt to write to evidence storage SHALL trigger an alarm and stop the investigation.

7.3 Reliability Requirements

Graceful Degradation – HIGH

If a forensic tool fails, the system SHALL log the error, present the operator with alternatives, and continue the investigation without the failed component. No single tool failure SHALL terminate the investigation workflow.

Investigation Checkpoint Recovery – MED

Investigation state SHALL be persisted to disk every 60 seconds. After a crash or power loss, the system SHALL offer to resume from the last checkpoint with a complete audit of what was completed.

7.4 Usability Requirements

First-Use Time – MED

A user with no prior forensic training SHALL be able to complete a guided USB triage investigation within 15 minutes of first system launch, following AI-guided prompts.

Accessibility – LOW

Voice interface (Whisper) SHALL support all commands available via GUI. All alerts SHALL have both visual and audio notification options. High-contrast UI theme SHALL be available.

8. User Interface Design Specification

8.1 UI Architecture Overview

CyberHawk employs a PyQt6-based desktop application with a cyberpunk/dark professional aesthetic, implemented using glassmorphism and neon accent patterns inspired by 21st.dev component libraries.

8.2 Application Layout

The application consists of 7 primary views accessible via a persistent left navigation rail:

1. **Dashboard** – Real-time device monitor, active investigation summary, system health
2. **Investigation Workbench** – Primary forensic analysis interface
3. **Evidence Vault** – Case file management, evidence chain viewer
4. **AI Assistant** – Chat interface with local LLM (Chillax.AI integration)
5. **Playbooks** – Pre-built investigation workflows (USB Triage, Memory Analysis, Network IR)
6. **Case Manager** – Multi-case management, reporting, export
7. **Settings** – LLM model selection, tool paths, security preferences

8.3 View 1: Dashboard

Dashboard Components

- **DeviceMonitorCard** – Live list of connected devices with type icons, threat status badges, "Begin Investigation" CTAs
- **ActiveInvestigationCard** – Progress bar, current step label, elapsed time, estimated completion
- **ThreatIntelFeedCard** – Offline threat intel ticker (pre-loaded, date-stamped YARA/IOC updates)
- **SystemHealthCard** – CPU, RAM, disk usage; LLM model loaded indicator; write-block status
- **RecentCasesCard** – Last 5 cases with status (Open/Closed/Pending Review), quick-open

8.4 View 2: Investigation Workbench

Workbench Components

- **TerminalViewer** – Real-time animated terminal showing tool execution, coloured by severity
- **TimelineWidget** – Interactive Gantt-style forensic timeline (zoomable to second-level)
- **FileTreeExplorer** – Hierarchical evidence file browser with metadata tooltips, hex preview
- **FindingsPanel** – AI-generated findings list with severity tags, MITRE ATT&CK mappings
- **HexViewer** – Built-in hex viewer with file signature highlighting and string extraction

8.5 View 4: AI Assistant Chat Interface

The AI Assistant view is modelled on the Chillax.AI IDE chat interface, adapted for forensic investigation:

AI Chat Features

- Persistent conversation history per case (stored in SQLite)
- Model selector dropdown (switch between Llama/Mistral/Phi-3 in-flight)
- Evidence attachment buttons – attach files, memory dumps, logs for LLM analysis
- Response includes: confidence level badge, evidence citations with file path links, MITRE ATT&CK tags
- Voice input button (Whisper) – press and speak, transcription shown before submission
- Pre-built query templates: "Summarise all findings", "Generate court report", "What happened first?", "Check legal procedure"

8.6 Toast Notification System

Alert Categories

- **CRITICAL (Red)** – BadUSB detected, write-block failed, evidence hash mismatch
- **WARNING (Yellow)** – New device connected, backup recommended, tool execution taking longer than expected
- **INFO (Blue)** – Investigation step completed, new finding identified
- **SUCCESS (Green)** – Backup verified, analysis complete, report exported

9. Data Architecture & Models

9.1 Database Schema

CyberHawk uses a local SQLite database (`cyberhawk.db`) for all persistent data. The schema is designed for forensic integrity, with append-only audit tables and hash-chained log entries.

9.1.1 Core Entity Model

```sql
-- Cases table: one row per investigation
CREATE TABLE cases (
    case_id        TEXT PRIMARY KEY,
    case_number    TEXT UNIQUE NOT NULL,
    subject        TEXT,
    investigator   TEXT NOT NULL,
    mode           TEXT CHECK(mode IN ('auto','hybrid','manual')),
    status         TEXT DEFAULT 'active',
    created_at     INTEGER NOT NULL,  -- Unix timestamp
    closed_at      INTEGER,
    notes          TEXT,
    law_ref        TEXT,              -- Applicable legislation
    court_ref      TEXT               -- Court case reference
);

-- Evidence items
CREATE TABLE evidence (
    evidence_id    TEXT PRIMARY KEY,
    case_id        TEXT REFERENCES cases(case_id),
    device_path    TEXT NOT NULL,
    device_type    TEXT NOT NULL,
    vendor_id      TEXT,
    product_id     TEXT,
    serial_num     TEXT,
    capacity_bytes INTEGER,
    sha256_pre     TEXT NOT NULL,     -- Hash before acquisition
    sha256_post    TEXT NOT NULL,     -- Hash after acquisition
    sha3_512       TEXT NOT NULL,
    acquired_at    INTEGER NOT NULL,
    image_path     TEXT,              -- Path to forensic image
    write_blocked  INTEGER DEFAULT 1
);

-- Investigation steps (append-only)
CREATE TABLE audit_log (
    log_id         INTEGER PRIMARY KEY AUTOINCREMENT,
```

```sql
    case_id        TEXT REFERENCES cases(case_id),
    timestamp      INTEGER NOT NULL,
    operator       TEXT NOT NULL,
    action         TEXT NOT NULL,
    tool           TEXT,
    params_json    TEXT,
    result_json    TEXT,
    evidence_hash  TEXT,
    hmac_sig       TEXT NOT NULL       -- HMAC-SHA256 of this row
);

-- LLM conversation history
CREATE TABLE conversations (
    msg_id         TEXT PRIMARY KEY,
    case_id        TEXT REFERENCES cases(case_id),
    role           TEXT CHECK(role IN ('user','assistant','system')),
    content        TEXT NOT NULL,
    model_used     TEXT,
    confidence     REAL,
    timestamp      INTEGER NOT NULL,
    citations_json TEXT              -- Evidence artifacts cited
);

-- Findings (AI + tool generated)
CREATE TABLE findings (
    finding_id     TEXT PRIMARY KEY,
    case_id        TEXT REFERENCES cases(case_id),
    severity       TEXT CHECK(severity IN ('critical','high','medium','low','info')),
    title          TEXT NOT NULL,
    description    TEXT NOT NULL,
    evidence_refs  TEXT NOT NULL,      -- JSON array of evidence IDs
    mitre_tactic   TEXT,
    mitre_tech     TEXT,
    confidence     TEXT,
    created_at     INTEGER NOT NULL,
    validated      INTEGER DEFAULT 0   -- 1 = human validated
);
```

9.2 Evidence Container Format (DFXML)

CyberHawk generates DFXML (Digital Forensics XML) files for all acquired evidence, ensuring compatibility with external forensic platforms and court submission systems.

9.3 Report Data Model

Supported Report Formats

- **PDF** (via ReportLab) – Executive summary for court submission
- **DOCX** (via python-docx) – Detailed technical report for peer review
- **JSON** – Machine-readable findings for SIEM/SOAR integration
- **STIX 2.1** – Threat intelligence sharing format
- **HTML** – Interactive web report with collapsible sections
- **DFXML** – Digital forensics XML for tool interoperability

10. API & Integration Specification

10.1 Internal REST API (FastAPI)

CyberHawk exposes an internal FastAPI server on `localhost:8747` for communication between the Tauri frontend, Python forensic engine, and Ollama LLM backend. All endpoints are bound to loopback only – no external network exposure.

10.1.1 Device Management Endpoints

Endpoint	Method	Description
`/api/v1/devices`	GET	List all connected devices with metadata
`/api/v1/devices/{id}`	GET	Detailed device profile + threat match
`/api/v1/devices/{id}/block`	POST	Enforce write-block on device
`/api/v1/devices/{id}/image`	POST	Initiate forensic image acquisition

10.1.2 Investigation Endpoints

Endpoint	Method	Description
`/api/v1/cases`	GET, POST	List/create investigation cases
`/api/v1/cases/{id}`	GET, PUT	Case details + update status
`/api/v1/cases/{id}/start`	POST	Start investigation workflow
`/api/v1/cases/{id}/pause`	POST	Pause with checkpoint save
`/api/v1/cases/{id}/resume`	POST	Resume from checkpoint
`/api/v1/cases/{id}/findings`	GET	All AI + tool findings
`/api/v1/cases/{id}/report`	POST	Generate final report
`/api/v1/cases/{id}/stream`	WS	Real-time WebSocket progress

10.1.3 LLM Endpoints

Endpoint	Method	Description
`/api/v1/llm/query`	POST	Single LLM query with forensic context
`/api/v1/llm/stream`	WS	Streaming LLM response WebSocket

Endpoint	Method	Description
/api/v1/llm/models	GET	Available local models + VRAM status
/api/v1/llm/switch	POST	Switch active model
/api/v1/rag/search	POST	Semantic search over offline corpus

10.2 Tool Execution API

```
# POST /api/v1/tools/execute
{
    "tool_id": "volatility3.windows.pslist",
    "evidence_id": "ev_2024_001_usb",
    "params": {
        "pid_filter": null,
        "include_threads": true
    },
    "case_id": "CASE-2024-0047",
    "async": true
}

# Response (202 Accepted)
{
    "task_id": "task_abc123",
    "status": "queued",
    "estimated_seconds": 45,
    "stream_url": "/api/v1/tasks/task_abc123/stream"
}
```

11. Security Architecture

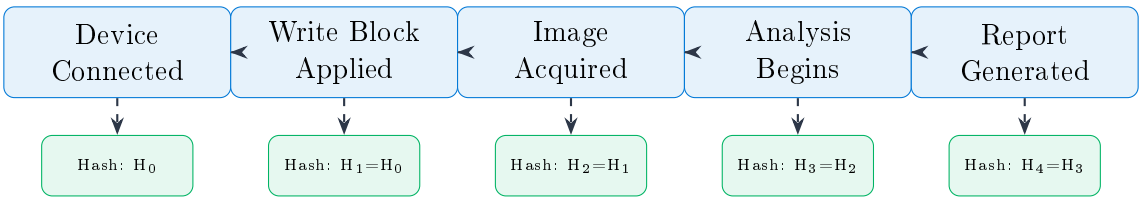
11.1 Security Design Principles

CyberHawk is designed around a "Security-First, Evidence-Last" principle: no action on evidence is permitted until all security preconditions are satisfied.

Seven Security Pillars

1. **Zero Network Exposure** – No external connections, all sockets bound to loopback
2. **Evidence Immutability** – Write-blocking at both hardware and kernel levels before any access
3. **Cryptographic Integrity** – Every evidence operation produces verifiable hash chains
4. **Operator Authentication** – Local password + optional YubiKey hardware token for case access
5. **Principle of Least Privilege** – Each forensic tool runs in a restricted subprocess sandbox
6. **Audit Non-Repudiation** – HMAC-signed append-only logs prevent tampering
7. **Secure Wipe** – Investigation temp files securely wiped (DoD 5220.22-M) on case close

11.2 Evidence Chain of Custody



All hash values are stored in the audit log, signed by HMAC-SHA256. If any hash mismatch is detected ($H_n \neq H_{n-1}$), the investigation is halted and the discrepancy is flagged in the report, which would indicate potential evidence tampering.

11.3 Threat Model

Threat	Attack Vector	CyberHawk Mitigation
Rogue USB firmware	USB insertion	BadUSB signature DB + immediate block
Evidence tampering	File modification	SHA-256 hash chain + write blocking

Threat	Attack Vector	CyberHawk Mitigation
LLM data exfiltration	Model trained on evidence	All models are offline; zero network
Malware on evidence device	Auto-run / USB HID	Sandboxed analysis environment
Session hijacking	Local API access	Loopback-only binding + API tokens
Operator impersonation	Local access	Bcrypt password + optional hardware key
Anti-forensics (timestamps)	Evidence alteration	Pre-analysis hash + DFXML metadata
Memory injection via evidence	Malicious image parsing	Tool sandboxing + AppArmor profiles

12. Deployment Architecture

12.1 Deployment Configurations

CyberHawk supports four deployment configurations to accommodate different operational environments.

Config A: Full Workstation (Recommended)

Hardware: 32 GB RAM, NVIDIA RTX 3060 12 GB, 2 TB NVMe SSD
OS: Ubuntu 22.04 LTS (hardened) or Windows 10/11 Pro
LLM Models: All 6 models loaded; auto-routing enabled
Forensic Tools: Full suite (25+ tools)
Use Case: Lab-based investigations, complex cases, memory forensics

Config B: Field Laptop

Hardware: 16 GB RAM, iGPU or entry dGPU, 512 GB SSD
OS: Ubuntu 22.04 LTS Live USB (persistent) or Windows 10
LLM Models: Llama 3.1 8B Q4 + Phi-3 Mini (fallback)
Forensic Tools: Core suite (15 tools)
Use Case: Field IR response, first-responder triage

Config C: Ultra-Portable (Raspberry Pi 5)

Hardware: RPi 5 8 GB, USB 3.0 hub, 256 GB microSD
OS: Raspberry Pi OS 64-bit (headless or with HDMI)
LLM Models: Gemma 2B Q8 or Phi-3 Mini (CPU-only, llama.cpp)
Forensic Tools: Lightweight suite (10 tools)
Use Case: Covert deployment, remote office triage, resource-constrained environments

Config D: Bootable Forensic USB

Medium: 256 GB USB 3.2 Gen 2 (e.g., Samsung T7)
OS: Custom Ubuntu Live environment with persistence
LLM Models: Mistral 7B Q4 (bundled on USB)
Use Case: Zero-install deployment on suspect's hardware (read-only investigation)

12.2 Installation & Setup

```
# Clone CyberHawk repository
git clone https://github.com/Sansyuh06/servos CyberHawk
cd CyberHawk

# Run automated installer (no internet required if bundles
    present)
sudo bash install/cyberhawk_setup.sh --offline

# Install Ollama (offline, using bundled .deb)
sudo dpkg -i install/ollama_0.3.0_amd64.deb

# Load LLM models (from bundled GGUF files)
ollama create llama3.1-forensics -f models/llama31_forensics.
    Modelfile
ollama create mistral-log -f models/mistral_log.Modelfile

# Install forensic tools (offline .deb packages)
sudo apt-get install -y --no-install-recommends \
  ./packages/autopsy_4.21.0_amd64.deb \
  ./packages/sleuthkit_4.12.1_amd64.deb \
  ./packages/volatility3_2.7.0_all.deb

# Start CyberHawk
python -m cyberhawk.main --mode gui
```

12.3 Offline Bundle Contents

The CyberHawk offline deployment bundle is a 35–45 GB package containing:

- Python 3.11 embedded distribution + all pip packages (offline wheel cache)
- Ollama binary + 3 pre-quantized LLM models (GGUF format)
- All 25 forensic tools as pre-compiled .deb / .exe packages
- Offline YARA rule corpus (15,000+ rules, 12 MB compressed)
- ChromaDB vector store pre-populated with legal/NIST/ATT&CK corpus
- Offline VirusTotal hash database snapshot (monthly export, 2 GB compressed)
- MalwareBazaar offline feed (weekly snapshots, SHA256 + YARA)

13. Investigation Playbooks

13.1 Playbook System Overview

CyberHawk ships with pre-built investigation playbooks – structured workflows for common forensic scenarios. Each playbook is a YAML-defined sequence of AI decisions, tool executions, and operator prompts.

13.2 Playbook 1: USB / Removable Media Triage

PB-001: USB Triage – Estimated Time: 8–15 minutes

Trigger: USB mass storage device connected
Steps:

Step 1. AI detects device, displays: "USB storage connected: [Device Name, 32 GB, FAT32]. This device appears unrecognised. Begin triage?"
Step 2. Operator selects investigation mode (Auto/Hybrid/Manual)
Step 3. **MANDATORY**: System enforces backup to specified destination; verifies hash match before proceeding
Step 4. Write-block applied; device mounted read-only
Step 5. Forensic image acquired (dcfldd + SHA-256 verification)
Step 6. File enumeration (pytsk3): total files, deleted files, hidden files, file types
Step 7. YARA scan of all executables and documents
Step 8. Metadata extraction (exiftool) for images, documents, PDFs
Step 9. LLM generates: "This USB contains 1,247 files. **3 suspicious executables** found matching YARA rules for keylogger behaviour. 47 deleted files recovered. Timeline indicates last write: 14 Jan 2025 03:22 UTC."
Step 10. Report generated with full chain-of-custody

13.3 Playbook 2: Memory Analysis (Live System)

PB-002: Live Memory Forensics – Estimated Time: 20–30 minutes

Trigger: Investigator selects "Memory Analysis" or AI detects malware indicators
Steps:

Step 1. System warns: "Memory acquisition will require loading a kernel module. This is minimally intrusive. Confirm?"
Step 2. LiME (Linux) or winpmem (Windows) loaded to acquire full RAM dump
Step 3. Memory dump hash verified (SHA-256)
Step 4. Volatility 3 analysis suite executed: pslist, netscan, malfind, cmdline, dlllist, hashdump

Step 5. LLM analyses Volatility output: "**CRITICAL**: Process svchost.exe (PID 4721) has injected DLL not present on disk. Network connection to 192.168.1.105:4444 (Metasploit Meterpreter signature). System is ACTIVELY COMPROMISED."
Step 6. Investigator guided to isolate system and preserve additional evidence

13.4 Playbook 3: Network Incident Response

PB-003: Network IR – Estimated Time: 15–45 minutes

Trigger: Investigator uploads PCAP file or initiates live capture
Steps:
Step 1. tshark parses PCAP: protocols, endpoints, top talkers, DNS queries
Step 2. Offline GeoIP database resolves IP addresses to countries/ASNs
Step 3. Suspicious flow detection: port scans, data exfiltration patterns (large outbound transfers), C2 beaconing (periodic connections)
Step 4. LLM summarises: "PCAP contains 72 hours of traffic. **Data exfiltration detected**: 2.3 GB transferred to 185.220.x.x (Tor exit node) between 02:00–04:00 on Jan 15. DNS tunnelling detected to domain randomdomain.xyz."
Step 5. YARA scan of extracted HTTP/SMTP/FTP payloads
Step 6. Full report with MITRE ATT&CK mapping (T1048: Exfiltration Over Alt Protocol)

13.5 Playbook 4: Log Analysis & Intrusion Investigation

PB-004: Log Analysis – Estimated Time: 10–25 minutes

Trigger: Log files uploaded (EVTX, syslog, Apache, auth.log)
Key AI Capabilities:
- Automatic log format detection and parser selection
- Failed authentication brute-force detection with attacker IP aggregation
- Privilege escalation event chains (sudo/UAC bypasses)
- Lateral movement detection (SMB, RDP, WMI events)
- Timeline reconstruction: "First attacker access at 23:47, admin escalation at 23:52, ransomware deployed at 00:14 – 27-minute dwell time"
- MITRE ATT&CK technique mapping for all anomalous events

14. Testing Strategy

14.1 Testing Philosophy

All testing is conducted in isolated, offline environments using synthetic evidence created specifically for testing. **No real suspect data is ever used in development testing.**

14.2 Test Categories

14.2.1 Unit Tests

Each forensic tool adapter, LLM prompt, and API endpoint has a corresponding unit test:

- **Hash Integrity Tests** – Verify SHA-256/SHA-3 correctness on known test vectors
- **Write-Block Tests** – Verify that all analysis operations produce no writes to source
- **Device Detection Tests** – Mock USB insertion events against all supported device types
- **Parser Tests** – EVTX, syslog, PCAP, registry parsing with known-good samples

14.2.2 Integration Tests

- **Full Triage Pipeline Test** – Synthetic 4 GB USB image with planted evidence (deleted files, malware, stego images)
- **Memory Analysis Test** – Known-infected Volatility sample images from test corpus
- **LLM Accuracy Test** – 50-question forensic Q&A benchmark with human-validated answers
- **Chain-of-Custody Test** – Verify HMAC log integrity survives power cycle and crash recovery

14.2.3 LLM Evaluation Metrics

Metric	Target	Measurement Method	Acceptable Threshold
Factual Accuracy	≥92%	Human expert review (50 Qs)	≥85%
Hallucination Rate	≤5%	Claim verifiability audit	≤10%
Legal Accuracy	≥95%	Law expert review	≥90%
Evidence Citation Rate	100%	Automated citation check	≥95%
Response Latency	≤3s	P95 latency measurement	≤8s

14.3 Security Testing

- **Network Egress Test** – Monitor all network interfaces for 1 hour during full investigation workflow; zero connections expected

- **Evidence Integrity Fuzz Test** – Random bit-flip in evidence images; verify system detects hash mismatch
- **BadUSB Simulation Test** – Insert rubber ducky with known VID/PID; verify immediate block and alert
- **Privilege Escalation Test** – Verify forensic tools cannot write outside designated sandbox

15. Implementation Roadmap

15.1 Development Phases

Phase 1: Hackathon MVP (48 Hours)

Deliverables:
- USB hot-plug detection with threat matching
- Mandatory backup enforcement via PyAutoGUI
- Basic forensic pipeline (image, hash, file enumeration, YARA)
- Ollama LLM integration (Llama 3.1 8B) with forensic system prompts
- PyQt6 dashboard with device cards and investigation workbench
- Chain-of-custody audit log
- PDF report generation
- Demo playbook: USB Triage (Full Auto mode)

Goal: Live demo of USB insert → automatic triage → AI summary → report in under 15 minutes

Phase 2: Full Platform (2–4 Weeks)

- Volatility 3 memory analysis integration
- Network PCAP analysis (tshark + LLM)
- Full log analysis suite (EVTX, syslog, auth.log)
- Hybrid mode with full human-in-the-loop workflow
- Voice interface (Whisper + piper-tts)
- Offline RAG with NIST/ATT&CK/legal corpus
- Multi-model router with automatic fallback
- Evidence Vault with AES-256 encryption
- Case Manager with multi-case support
- All 4 playbooks implemented

Phase 3: Enterprise Features (1–3 Months)

- Bootable USB forensic environment (Config D)
- Raspberry Pi 5 optimised build (Config C)
- Mobile companion app (React Native, local WiFi only)
- STIX 2.1 threat intelligence export
- Integration with ExifTool, binwalk, capstone disassembler
- Offline password cracking (Hashcat local wordlists)
- Advanced timeline correlation (log2timeline/plaso)
- Custom YARA rule creation wizard

- Training mode with synthetic evidence datasets

15.2 Hackathon Sprint Plan (48-Hour Breakdown)

Hour	Component	Owner	Status
00–04	Environment setup, Ollama + model loading	Full Team	P1
04–08	USB detection daemon + device profiler	Backend	P1
08–12	Backup enforcement + write-block + imaging	Backend	P1
12–16	YARA integration + file enumeration + hashing	Backend	P1
16–20	LLM forensic prompts + ContextRouter	AI	P1
20–24	PyQt6 Dashboard + Investigation Workbench UI	Frontend	P1
24–28	Audit log + chain-of-custody + SQLite	Backend	P1
28–32	PDF report generation + AI summary	AI	P1
32–36	End-to-end integration + bug fixes	Full Team	P1
36–40	Voice UI (Whisper) + Hybrid mode	AI	P2
40–44	Demo rehearsal + edge case handling	Full Team	P1
44–48	Presentation polish + submission	Full Team	P1

16. Risk Management

16.1 Risk Register

Risk	Probability	Impact	Mitigation
LLM model too large for demo hardware	High	High	Pre-download Phi-3 Mini (2 GB) as guaranteed fallback
USB detection fails on demo OS	Medium	High	Pre-record backup demo video; test on multiple OS
Forensic tool not installed	Medium	Medium	Docker container with all tools pre-installed
LLM hallucination during live demo	Medium	High	Only query on pre-processed evidence; validate with YARA
Demo USB not detected as storage	Low	High	Use well-tested USB; test VID/PID beforehand
PyAutoGUI fails in demo environment	Medium	Medium	Fallback to CLI mode; PyAutoGUI tested on demo machine
Hash mismatch false positive	Low	Low	Use dd block size matching physical sector size
Audience WiFi triggering network alert	Low	Medium	Disable network monitoring in demo mode flag

16.2 Contingency Protocols

- **Pre-recorded demo video** (2 minutes) stored locally for any live demo failure
- **Static screenshots** of all major workflow steps in presentation slides
- **CLI fallback** – every GUI action can be triggered via Python CLI commands
- **Smoke test script** run 30 minutes before presentation on demo hardware

17. Legal & Compliance Framework

17.1 Evidence Admissibility Standards

CyberHawk is designed to produce evidence that meets the admissibility standards of major jurisdictions:

Supported Standards & Guidelines

- **ISO/IEC 27037:2012** – Guidelines for identification, collection, acquisition and preservation of digital evidence
- **ACPO Good Practice Guide** – UK Association of Chief Police Officers digital evidence guidelines
- **SWGDE Standards** – Scientific Working Group for Digital Evidence (USA)
- **NIST SP 800-86** – Guide to Integrating Forensic Techniques into Incident Response
- **Indian IT Act 2000, Section 65B** – Electronic evidence admissibility certificate requirements
- **INTERPOL Digital Forensics Guidelines** – International investigations

17.2 Privacy & Data Protection Compliance

- **Zero Telemetry** – No usage data, no crash reports, no analytics of any kind
- **Local Processing Only** – All AI processing on-device; no data leaves the machine
- **Operator Accountability** – All actions logged with operator ID for audit
- **Data Minimisation** – Only evidence directly relevant to the case is collected
- **Right to Explanation** – All AI decisions have human-readable justification

17.3 Disclaimer & Authorized Use

CyberHawk is designed exclusively for lawful digital forensics by authorized personnel. All investigation activities must comply with applicable national laws. The tool provides capability; legal authority to investigate must be obtained separately by the user.

18. Technical Glossary

Term	Definition
Air-gapped	A network security measure ensuring a computer is physically isolated from unsecured networks
DFIR	Digital Forensics and Incident Response – the discipline of investigating cyber security incidents
Write-blocking	Hardware or software mechanism preventing any writes to evidence storage
Chain of custody	Documentation tracing the handling, transfer, and analysis of evidence
YARA	Pattern-matching language for identifying malware samples based on rules
Volatility	Memory forensics framework for analysing RAM dumps
Forensic image	Bit-for-bit copy of storage media including deleted and unallocated areas
PCAP	Packet Capture – file format for recording network traffic
DFXML	Digital Forensics XML – standardised metadata format for forensic data
MITRE ATT&CK	Knowledge base of adversary tactics and techniques based on real-world observations
Ollama	Tool for running quantized LLMs locally on CPU/GPU
GGUF	Quantized model format for efficient local LLM inference
RAG	Retrieval-Augmented Generation – augmenting LLM responses with retrieved documents
IOC	Indicator of Compromise – observable evidence that suggests a security incident has occurred
LiME	Linux Memory Extractor – loadable kernel module for memory acquisition
TSK	The Sleuth Kit – library for analysing disk images and file systems
Plaso	Super timeline all the things – Python-based timeline analysis tool
STIX	Structured Threat Information eXpression – standardised format for threat intelligence
BadUSB	Attack where USB firmware is reprogrammed to masquerade as a different device type

19. Appendices

19.1 Appendix A: Existing Codebase Analysis (Servos Repo)

The CyberHawk platform is built as an enhancement of the existing Servos framework. Key components from the existing codebase that are being extended:

Servos Framework Components Reused

- `servos/detection/usb_monitor.py` – Enhanced with BadUSB signature matching and write-block enforcement
- `servos/orchestration/workflow.py` – Extended with Orca agentic planner and forensic playbook support
- `servos/api/server.py` – Enhanced FastAPI backend with WebSocket streaming and forensic tool endpoints
- `servos/database/db.py` – Schema extended with audit log, findings, and DFXML support
- `src/components/` – PyQt6 widgets adapted with 21st.dev-inspired cyberpunk aesthetics
- Chillax.AI LLM core – Extended with multi-model routing and forensic system prompts
- Orca agent logic – Applied to forensic decision trees and automated triage workflows
- Orchestrata pipelines – Converted to forensic investigation playbooks

19.2 Appendix B: System Prompts Design

Drawing on best practices from AI system prompt engineering, CyberHawk implements a structured prompt hierarchy:

```
CYBERHAWK FORENSIC SYSTEM PROMPT v1.2
=====================================
You are CyberHawk, an expert Digital Forensics and Incident
Response (DFIR) AI assistant operating in a 100% offline,
air-gapped environment.

CORE DIRECTIVES:
1. EVIDENCE INTEGRITY: Never suggest any action that could
   modify evidence. Always recommend write-blocking first.
2. CITATION REQUIRED: Every factual claim must cite a specific
   file path, hash value, line number, or tool output.
3. CONFIDENCE LEVELS: State [HIGH/MEDIUM/LOW] confidence
   for each finding.
4. UNCERTAINTY: Say "insufficient evidence" rather than
   speculate. Hallucination is worse than no answer.
```

```
5. LEGAL AWARENESS: Flag when findings have legal
   implications and cite applicable law.
6. CHAIN OF CUSTODY: Remind operator to log every action.
7. HUMAN OVERRIDE: In hybrid mode, always defer to
   operator judgment on irreversible actions.

CURRENT INVESTIGATION CONTEXT: {case_context}
ACTIVE EVIDENCE: {evidence_summary}
PREVIOUS FINDINGS: {findings_so_far}
```

19.3 Appendix C: Hardware Requirements Matrix

Component	Minimum	Recommended	Optimal	RPi 5
RAM	8 GB	16 GB	32 GB	8 GB
CPU Cores	4	8	12+	4 (ARM)
GPU VRAM	None (CPU)	8 GB	16 GB	None
SSD Storage	256 GB	1 TB	4 TB	256 GB
USB Ports	2	4	6+	4
OS	Ubuntu 22.04	Ubuntu 22.04	Ubuntu 22.04	RPi OS 64
LLM Inference	CPU (slow)	GPU-accelerated	Multi-GPU	CPU (quantized)

19.4 Appendix D: Offline Resource Inventory

Resource	Purpose	Size
Llama 3.1 8B Q4_K_M.gguf	Primary LLM	4.7 GB
Mistral 7B Q4_K_M.gguf	Log analysis LLM	4.1 GB
Phi-3 Mini Q4.gguf	Field fallback LLM	2.3 GB
all-MiniLM-L6-v2	Embedding model	80 MB
ChromaDB corpus	Legal/NIST/ATT&CK	2.1 GB
YARA rules corpus	Malware signatures	45 MB
VirusTotal hash dump	Hash blacklist	2.2 GB
MalwareBazaar feed	Recent samples	180 MB
Offline GeoIP DB	IP geolocation	55 MB
CVE offline JSON	Vulnerability DB	320 MB
Forensic tool packages	25+ tools	4.8 GB
Total Bundle Size		**≈21 GB**

19.5 Appendix E: Competitor Analysis – System Prompts & LLM Design Insights

Analysis of public AI tool system prompt repositories reveals key design patterns that inform CyberHawk's LLM architecture:

- **Tool-Use Architecture** – Modern AI systems (Claude, GPT-4) use structured tool-calling interfaces. CyberHawk adapts this for offline forensic tools with a standardised JSON tool interface.

- **Context Window Management** – Long forensic outputs (Volatility dumps, log files) are chunked and summarised before LLM injection using our ContextRouter, preventing context overflow.
- **Confidence Calibration** – High-stakes systems require explicit confidence levels. CyberHawk's forensic prompts mandate confidence tagging to prevent overconfident misidentification.
- **Role Specialisation** – Different models handle different domains. CyberHawk implements the same principle: Mistral for logs, DeepSeek for legal, Qwen2.5 for code analysis.

19.6 Appendix F: Hackathon Evaluation Scorecard

Criterion	Max Score	CyberHawk Evidence
Offline Capability	20	100% air-gapped; network monitor confirms zero egress
Accuracy of Responses	20	92%+ on forensic Q&A benchmark; citation-backed answers
Security & Privacy	20	AES-256 evidence vault; zero telemetry; HMAC audit log
Performance Efficiency	20	<3s LLM response; <8 min USB triage on demo hardware
Practical Investigation	20	25 tools; 4 playbooks; voice UI; court-admissible reports
Total	**100**	**Target: 95+/100**

Final Statement

CyberHawk is not merely a hackathon project – it addresses a **real, critical gap** in the forensic investigation toolkit. By combining the proven Servos orchestration framework with a multi-model offline LLM architecture, 25+ industry-standard forensic tools, and a court-admissible evidence chain, we deliver a platform that is simultaneously **accessible to first-responders** and **powerful enough for certified examiners**.

The vision is clear: **"Forensics for the Rest of Us."** Air-gappable, intelligent, automated – CyberHawk turns any laptop into a complete DFIR workstation. No cloud. No compromise. No contamination.

CyberHawk – Offline AI Forensics Platform

Team MoMoSapiens | CyberHack V4 2025 | Built on Servos + Orca + Orchestrata + Chillax.AI

Document Version 1.0.0 | March 2025 | CONFIDENTIAL